Name – Sangram Lembe

Reg No – RA2512051010001 (Data Eng.)

#Lab 9a) Building a Machine Learning Microservice using Docker

Step 1: mkdir ~/fraud-detection-demo: Used to create a directory called “fraud-detection-demo”

Step 2: from the directory, create an app.py file and add the code logic.

dockersteps | app.py | Dockerfile

```python
from fastapi import FastAPI
from pydantic import BaseModel
import uvicorn

app = FastAPI()

# This defines exactly what the "Amount" field should look like
class Transaction(BaseModel):
    amount: float

@app.get("/")
def home():
    return {"status": "ML Model API is live on Ubuntu!"}

@app.post("/predict")
def predict(data: Transaction):
    # User data.amount instead of a dictionary lookup
    result = "Fraud" if data.amount > 1000 else "Legit"
    return {"transaction_amount": data.amount, "prediction": result}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

Step 3: create a requirements.txt file and add required packages that need to be installed.

requirements.txt

```
fastapi
uvicorn
```

Step 4: create dockerfile and add docker code to it.

dockersteps | app.py | Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python3", "app.py"]
```

Step 5:

Update the package list

sudo apt update

Install Docker

sudo apt install docker.io -y

Start the Docker service

sudo systemctl start docker

Make sure Docker starts every time you reboot the VM

sudo systemctl enable docker

Add your current user to the docker group

sudo usermod -aG docker $USER

“after this step, logout and re-login again the ubuntu machine or clode the current terminal and open a new terminal.”

step 6:inside the directory, build the docker image.

```
hadoop@BDTT:~$ cd ~/fraud-detection-demo
hadoop@BDTT:~/fraud-detection-demo$ sudo docker build -t fraud-model:v1 .
[sudo] password for hadoop:
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.608kB
Step 1/7 : FROM python:3.9-slim
3.9-slim: Pulling from library/python
38513bd72563: Pulling fs layer
b3ec39b36ae8: Pulling fs layer
fc7443084902: Pulling fs layer
ea56f685404a: Pulling fs layer
ea56f685404a: Waiting
b3ec39b36ae8: Verifying Checksum
b3ec39b36ae8: Download complete
fc7443084902: Verifying Checksum
fc7443084902: Download complete
38513bd72563: Verifying Checksum
38513bd72563: Download complete
ea56f685404a: Verifying Checksum
ea56f685404a: Download complete
38513bd72563: Pull complete
b3ec39b36ae8: Pull complete
fc7443084902: Pull complete
ea56f685404a: Pull complete
Digest: sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
```

Step 7: run the container and install curl.

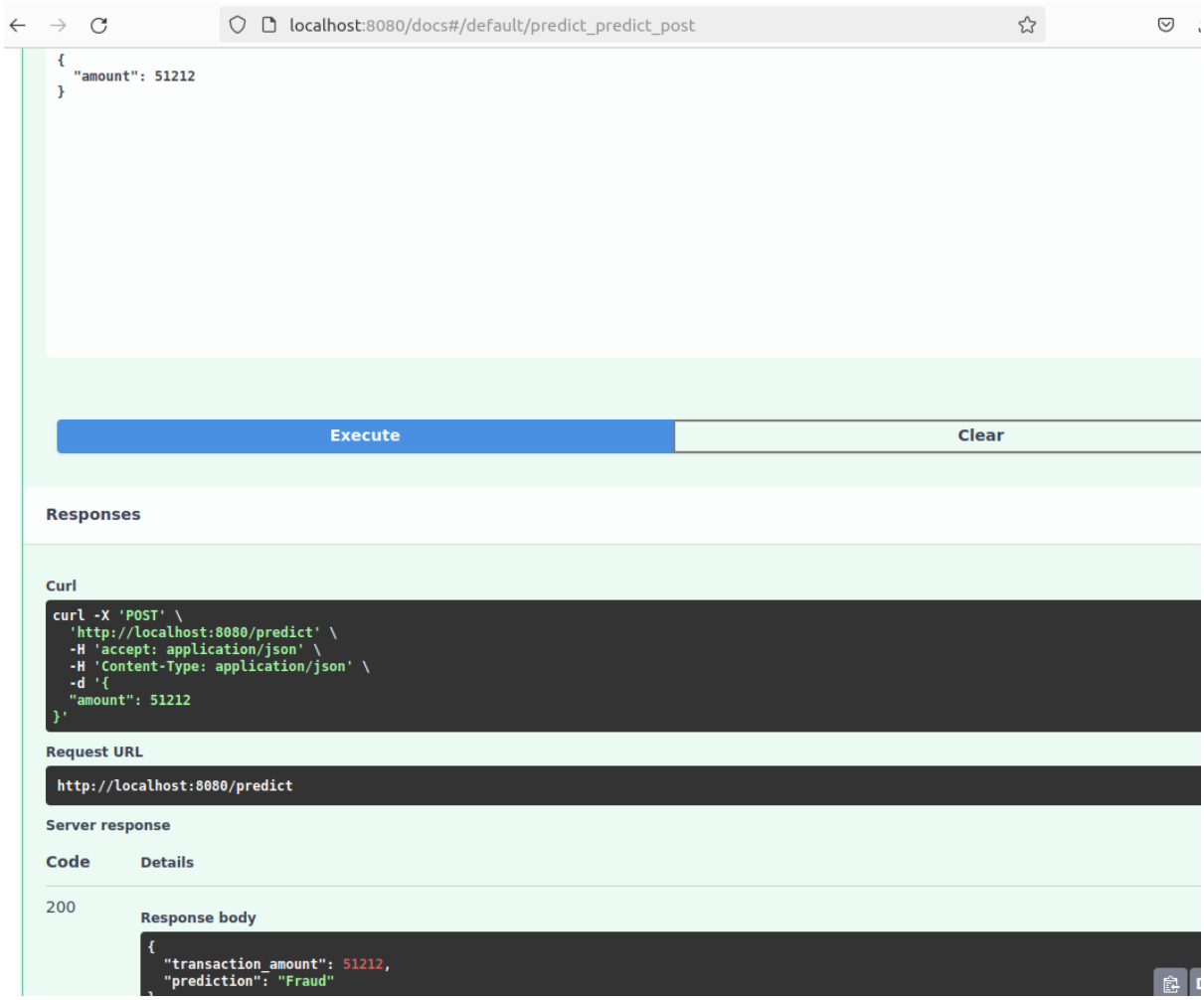
```
hadoop@BDTT:~/fraud-detection-demo$ sudo docker run -d --name fraud-service -p 8080:8080 fraud-model:v1
ac6a4e79fe7307fb50709ea656c4855748fa783acb13a17e7b6452b5343fff41
hadoop@BDTT:~/fraud-detection-demo$ sudo apt update && sudo apt install curl -y
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
475 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Step 8:

```
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.6) ...
hadoop@BDTT:~/fraud-detection-demo$ curl -X POST http://localhost:8080/predict \-H "Content-Type: application/json" \
-d '{"amount": 1500}'
hadoop@BDTT:~/fraud-detection-demo$
```

Expected output: {"transaction_amount":1500,"prediction":"Fraud"}

Alternate way of viewing output in localhost:8080



Since the amount=51212 which is greater than 1000. We get output as “Fraud”.

#Lab 9b) Building a MLflow and Docker integrated application

Step 1: mkdir ~/fraud-detection-demo: Used to create a directory called “fraud-detection-demo”

Step 2: create train.py and add the code logic to it. Cat command is used to add content to the file.

```
hadoop@BDTT: ~/fraud-detection-demo                    ×    hadoop@BDTT: ~/fraud-detecti
hadoop@BDTT:~$ cd ~/fraud-detection-demo
hadoop@BDTT:~/fraud-detection-demo$ touch train.py
hadoop@BDTT:~/fraud-detection-demo$ cat> train.py
import mlflow
import pickle

# 1. Setup: Tell MLflow where to store the data locally
mlflow.set_tracking_uri("http://0.0.0.0:5000")
mlflow.set_experiment("Fraud_Detection_Research")

def run_training_experiment(threshold_value, model_name):
    # Start a new "Run" in the librarian's logbook
    with mlflow.start_run(run_name=model_name):
        # Log the setting (Hyperparameter)
        mlflow.log_param("fraud_limit", threshold_value)

        # Simulate an accuracy score (Metric)
        # Usually, a tighter limit ($500) might catch more fraud but be less 'accurate'
        accuracy = 0.95 if threshold_value == 1000 else 0.82
        mlflow.log_metric("accuracy", accuracy)

        # Log a "Tag" to know who ran this
        mlflow.set_tag("stage", "testing")

        with open("model.pkl", "wb") as f:
            pickle.dump(threshold_value, f) # saving the threshold as our 'model'
            mlflow.log_artifact("model.pkl") # This uploads the model file to Mlflow
            print(f"Experiment {model_name} logged with accuracy: {accuracy}")

if __name__ == "__main__":
    # Simulate two different experiments
    run_training_experiment(1000, "Standard_Model")
    run_training_experiment(500, "Strict Model")
```

Step 3: install mlflow.

```
hadoop@BDTT:~/fraud-detection-demo$ mlflow ui --host 0.0.0.0 &
[1] 10174
hadoop@BDTT:~/fraud-detection-demo$ Backend store URI not provided. Using sqlite:///mlflow.db
Registry store URI not provided. Using backend store URI.
2026/03/23 09:01:26 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/03/23 09:01:26 INFO mlflow.store.db.utils: Updating database tables
[MLflow] Security middleware enabled with default settings (localhost-only). To allow connections from other hosts, use --host 0.0.0
.0 and configure --allowed-hosts and --cors-allowed-origins.
INFO:     Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
INFO:     Started parent process [10189]
2026/03/23 09:01:29 INFO mlflow.server.jobs.utils: Starting huey consumer for job function run_online_trace_scorer
2026/03/23 09:01:29 INFO mlflow.server.jobs.utils: Starting huey consumer for job function run_online_session_scorer
2026/03/23 09:01:30 INFO mlflow.server.jobs.utils: Starting huey consumer for job function optimize_prompts
2026/03/23 09:01:30 INFO mlflow.server.jobs.utils: Starting huey consumer for job function invoke_scorer
2026/03/23 09:01:30 INFO mlflow.server.jobs.utils: Starting dedicated Huey consumer for periodic tasks
INFO:     Started server process [10195]
INFO:     Waiting for application startup.
```

Step 4:

```
hadoop@BDTT:~/fraud-detection-demo$ pip install mlflow
Defaulting to user installation because normal site-packages is not writeable
Collecting mlflow
  Downloading mlflow-3.10.1-py3-none-any.whl (10.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.2/10.2 MB 7.4 MB/s eta 0:00:00
Collecting huey<3,>=2.5.4
  Downloading huey-2.6.0-py3-none-any.whl (76 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 77.0/77.0 KB 7.5 MB/s eta 0:00:00
Collecting mlflow-skinny==3.10.1
  Downloading mlflow_skinny-3.10.1-py3-none-any.whl (3.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.0/3.0 MB 20.2 MB/s eta 0:00:00
Collecting Flask<4
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
```

Step 5:

dockersteps(1) × app.py ×

```python
import pickle
from fastapi import FastAPI
import uvicorn
from pydantic import BaseModel

app = FastAPI()

# 1. Load the "Winning" recipe we saved in Demo 2
with open("model.pkl", "rb") as f:
    best_threshold = pickle.load(f)

class Transaction(BaseModel):
    amount: float

@app.post("/predict")
def predict(data: Transaction):
    # Now we use the threshold we got from our MLflow experiments!
    result = "Fraud" if data.amount > best_threshold else "Legit"
    return {
        "prediction": result,
        "threshold_used": best_threshold
    }
print("Starting the Fraud Detection Server...")
uvicorn.run(app, host="0.0.0.0", port=8080)
```

requirements.txt

```
fastapi
uvicorn
mlflow
```

train.py | requirements.txt | dockersteps(

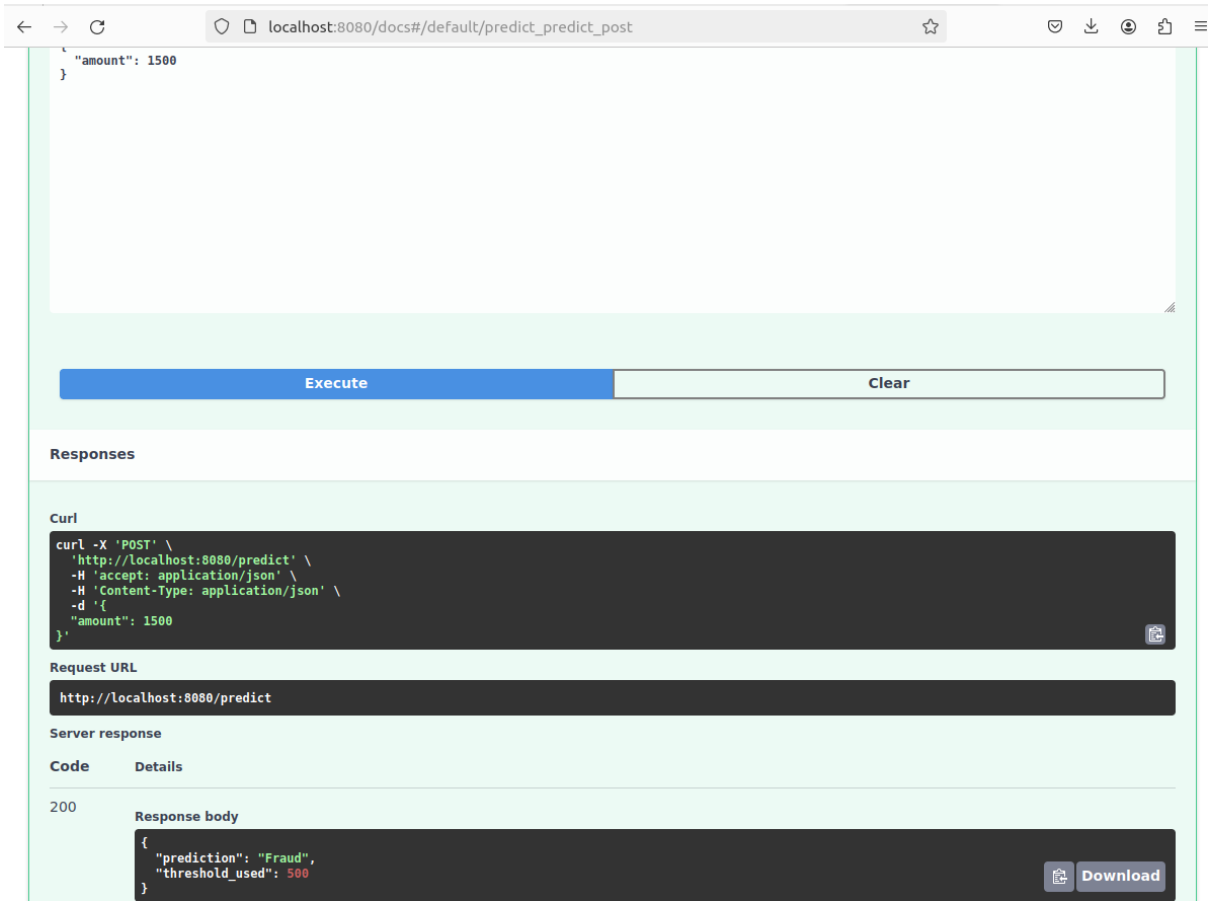
```python
import mlflow
import pickle

# 1. Setup: Tell MLflow where to store the data locally
mlflow.set_tracking_uri("http://0.0.0.0:5000")
mlflow.set_experiment("Fraud_Detection_Research")

def run_training_experiment(threshold_value, model_name):
    # Start a new "Run" in the librarian's logbook
    with mlflow.start_run(run_name=model_name):
        # Log the setting (Hyperparameter)
        mlflow.log_param("fraud_limit", threshold_value)

        # Simulate an accuracy score (Metric)
        # Usually, a tighter limit ($500) might catch more fraud but be less 'accurate'
        accuracy = 0.95 if threshold_value == 1000 else 0.82
        mlflow.log_metric("accuracy", accuracy)

        # Log a "Tag" to know who ran this
        mlflow.set_tag("stage", "testing")

        with open("model.pkl", "wb") as f:
            pickle.dump(threshold_value, f) # saving the threshold as our 'model'
            mlflow.log_artifact("model.pkl") # This uploads the model file to Mlflow
            print(f"Experiment {model_name} logged with accuracy: {accuracy}")

if __name__ == "__main__":
    # Simulate two different experiments
    run_training_experiment(1000, "Standard_Model")
    run_training_experiment(500, "Strict_Model")
```

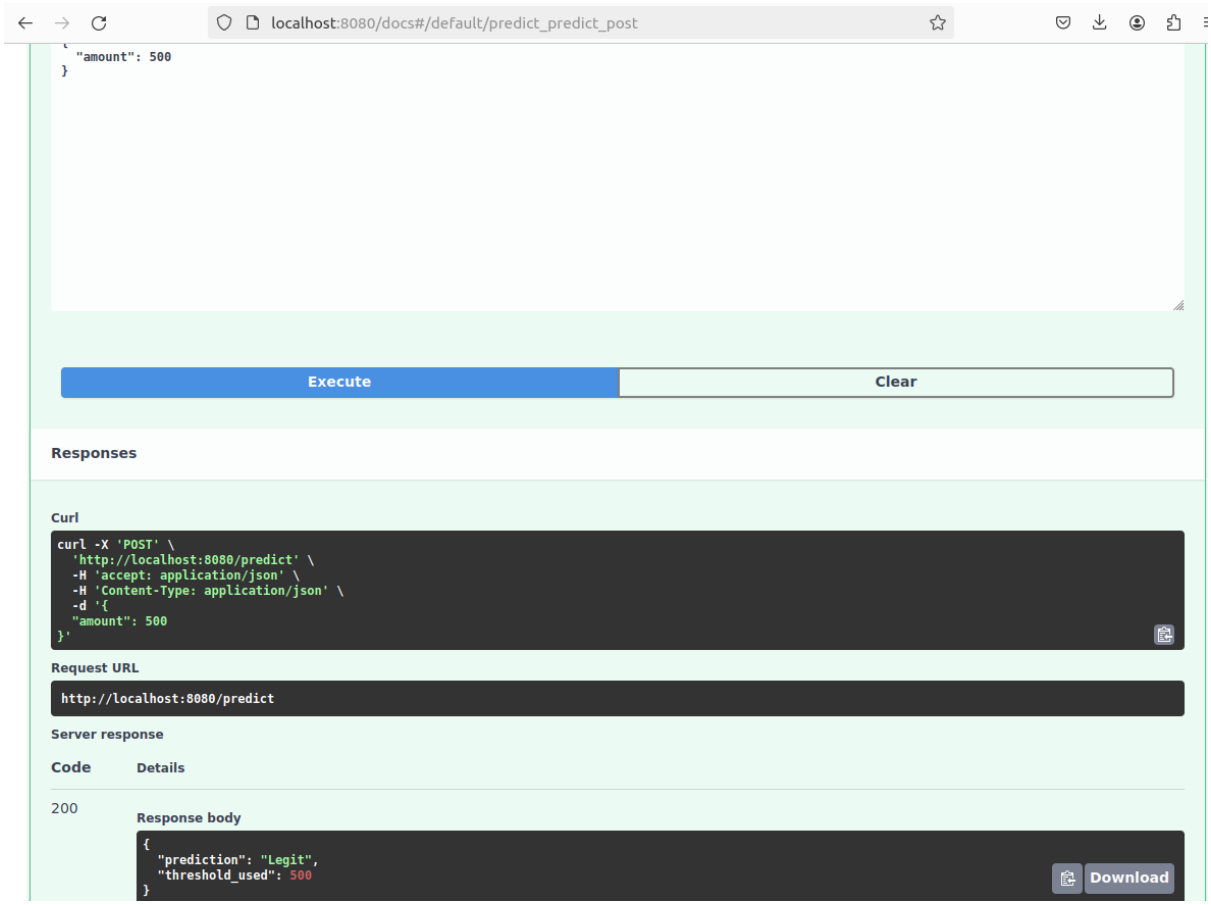
Step 6:

```
hadoop@BDTT:~/fraud-detection-demo$ python3 train.py
2026/03/23 09:09:30 INFO mlflow.tracking.fluent: Experiment with name 'Fraud_Detection_Research' does not exist. Creating a new expe
riment.
Experiment Standard_Model logged with accuracy: 0.95
🏃 View run Standard_Model at: http://0.0.0.0:5000/#/experiments/1/runs/3b5e1031f9714120948df63a7d428954
🧪 View experiment at: http://0.0.0.0:5000/#/experiments/1
Experiment Strict_Model logged with accuracy: 0.82
🏃 View run Strict_Model at: http://0.0.0.0:5000/#/experiments/1/runs/4759734fa32d4217b488d1ffe3c0e650
🧪 View experiment at: http://0.0.0.0:5000/#/experiments/1
hadoop@BDTT:~/fraud-detection-demo$ ls
app.py      mlartifacts  model.pkl         train.py
Dockerfile  mlflow.db    requirements.txt
hadoop@BDTT:~/fraud-detection-demo$ docker rm -f fraud-service || true
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Delete "http://%2Fvar%2Frun%2F
docker.sock/v1.50/containers/fraud-service?force=1": dial unix /var/run/docker.sock: connect: permission denied
hadoop@BDTT:~/fraud-detection-demo$ ^C
hadoop@BDTT:~/fraud-detection-demo$ sudo docker rm -f fraud-service
[sudo] password for hadoop:
fraud-service
hadoop@BDTT:~/fraud-detection-demo$ docker rm -f fraud-service || true
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Delete "http://%2Fvar%2Frun%2F
docker.sock/v1.50/containers/fraud-service?force=1": dial unix /var/run/docker.sock: connect: permission denied
hadoop@BDTT:~/fraud-detection-demo$ sudo usermod -aG docker $USER
hadoop@BDTT:~/fraud-detection-demo$ docker rm -f fraud-service || true
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Delete "http://%2Fvar%2Frun%2F
docker.sock/v1.50/containers/fraud-service?force=1": dial unix /var/run/docker.sock: connect: permission denied
hadoop@BDTT:~/fraud-detection-demo$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdoc
ker.sock/v1.50/containers/json": dial unix /var/run/docker.sock: connect: permission denied
hadoop@BDTT:~/fraud-detection-demo$ newgrp docker
hadoop@BDTT:~/fraud-detection-demo$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
hadoop@BDTT:~/fraud-detection-demo$ docker build -t fraud-model:final .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/
```

Step 7:we can check the fraud/legit using localhost:8080

```
hadoop@BDTT:~/fraud-detection-demo$ docker run --name fraud-service -p 8080:8080 fraud-model:final
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
Starting the Fraud Detection Server...
INFO:     172.17.0.1:48808 - "POST /predict HTTP/1.1" 200 OK
INFO:     172.17.0.1:56664 - "GET /docs HTTP/1.1" 200 OK
INFO:     172.17.0.1:56664 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     172.17.0.1:38064 - "POST /predict HTTP/1.1" 200 OK
```


localhost:8080/docs#/default/predict_predict_post
{
"amount": 1500
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://localhost:8080/predict' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"amount": 1500
}'
Request URL
http://localhost:8080/predict
Server response
Code
Details
200
Response body
{
"prediction": "Fraud",
"threshold_used": 500
}
Download


localhost:8080/docs#/default/predict_predict_post
{
"amount": 500
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://localhost:8080/predict' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"amount": 500
}'
Request URL
http://localhost:8080/predict
Server response
Code
Details
200
Response body
{
"prediction": "Legit",
"threshold_used": 500
}
Download

Step 8 we can check in terminal using:

```
hadoop@BDTT:~/fraud-detection-demo$ curl -X POST http://localhost:8080/predict \
-H "Content-Type: application/json" \
-d '{"amount": 1200}'
{"prediction":"Fraud","threshold_used":500}hadoop@BDTT:~/fraud-detection-demo$
```

Lab 9c) Using a real ML model instead of hardcoded number in pickle file.

Update app.py, train.py, requirements.txt

dockersteps × app.py × train.py

```python
import mlflow.pyfunc
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel
import uvicorn

app = FastAPI()

# Load model
model = mlflow.pyfunc.load_model("./iris_model")

class IrisInput(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

@app.post("/predict")
def predict(data: IrisInput):
    # Convert to DataFrame (IMPORTANT)
    input_df = pd.DataFrame([{
        "sepal_length": data.sepal_length,
        "sepal_width": data.sepal_width,
        "petal_length": data.petal_length,
        "petal_width": data.petal_width
    }])

    prediction = model.predict(input_df).tolist()

    return {"prediction": prediction}

print("Starting Iris Model API...")
uvicorn.run(app, host="0.0.0.0", port=8080)
```

dockersteps × app.py × train.py ×

```python
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import pandas as pd
import os

# 1. Load Real Data
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2)

# 2. Start MLflow Experiment
mlflow.set_experiment("Iris_Classification_System")

with mlflow.start_run():
    # 3. Train a Real Model
    clf = RandomForestClassifier(n_estimators=100)
    clf.fit(X_train, y_train)

    # 4. Log Metrics & Model to MLflow
    accuracy = clf.score(X_test, y_test)
    mlflow.log_metric("accuracy", accuracy)

    # Save the model as a local file for Docker to grab
    mlflow.sklearn.save_model(clf, "iris_model", serialization_format="cloudpickle")
    print(f"Model trained with accuracy: {accuracy}")

```

dockersteps | app.py | train.py | requirements.txt

```
1 fastapi
2 uvicorn
3 mlflow
4 scikit-learn
5 pandas
```

Install mlflow , fastapi, uvicorn:

```
hadoop@BDTT:~/fraud-detection-demo$ pip install mlflow fastapi uvicorn
Defaulting to user installation because normal site-packages is not writeable
Collecting mlflow
  Downloading mlflow-3.10.1-py3-none-any.whl (10.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.2/10.2 MB 5.0 MB/s eta 0:00:00
Requirement already satisfied: fastapi in /home/hadoop/.local/lib/python3.10/site-packages (0.117.1)
Requirement already satisfied: uvicorn in /home/hadoop/.local/lib/python3.10/site-packages (0.41.0)
Collecting numpy<3
  Downloading numpy-2.2.6-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.8/16.8 MB 5.1 MB/s eta 0:00:00
Collecting scipy<2
  Downloading scipy-1.15.3-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (37.7 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 37.7/37.7 MB 4.5 MB/s eta 0:00:00
Collecting docker<8,>=4.0.0
  Downloading docker-7.1.0-py3-none-any.whl (147 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 147.8/147.8 KB 4.3 MB/s eta 0:00:00
Collecting mlflow-skinny==3.10.1
  Downloading mlflow_skinny-3.10.1-py3-none-any.whl (3.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.0/3.0 MB 5.1 MB/s eta 0:00:00
Collecting Flask-CORS<7
  Downloading flask_cors-6.0.2-py3-none-any.whl (13 kB)
Collecting skops<1
```

```
hadoop@BDTT:~/fraud-detection-demo$ mlflow ui --host 0.0.0.0 > mlflow.log 2>&1 &
[1] 9500
hadoop@BDTT:~/fraud-detection-demo$ python3 train.py
2026/03/27 11:07:13 INFO mlflow.tracking.fluent: Experiment with name 'Iris_Classification_System' does not exist. Creating a new ex
periment.
2026/03/27 11:07:14 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising cauti
on because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. T
he recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.
html
Model trained with accuracy: 1.0
hadoop@BDTT:~/fraud-detection-demo$ ls
app.py      iris_model  mlflow.log        train.py
Dockerfile  mlflow.db   requirements.txt
hadoop@BDTT:~/fraud-detection-demo$ ls iris_model
conda.yaml  model.pkl        requirements.txt
MLmodel     python_env.yaml
hadoop@BDTT:~/fraud-detection-demo$ docker rm -f fraud-service || true
fraud-service
hadoop@BDTT:~/fraud-detection-demo$ docker build -t iris-model:final .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  822.8kB
Step 1/7 : FROM python:3.9-slim
 ---> 085da638e1b8
Step 2/7 : WORKDIR /app
 ---> Using cache
```

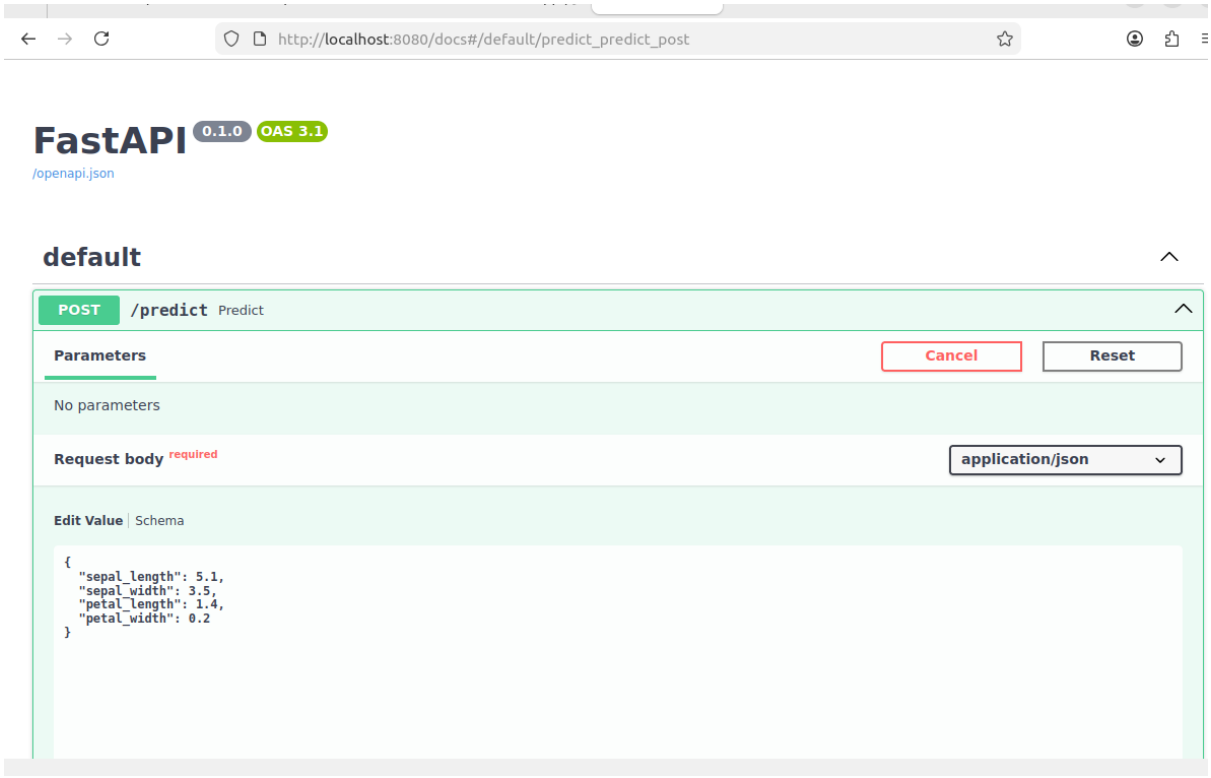
```
hadoop@BDTT:~/fraud-detection-demo$ docker rm -f iris-service || true
iris-service
hadoop@BDTT:~/fraud-detection-demo$ docker build -t iris-model:final .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  828.4kB
Step 1/7 : FROM python:3.9-slim
 ---> 085da638e1b8
Step 2/7 : WORKDIR /app
 ---> Using cache
 ---> ff3ceb0e0d8e
Step 3/7 : COPY requirements.txt .
 ---> 90bb093e5d4d
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
 ---> Running in 0aa90b047557
Collecting fastapi
  Downloading fastapi-0.128.8-py3-none-any.whl (103 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 103.6/103.6 kB 4.0 MB/s eta 0:00:00
Collecting uvicorn
  Downloading uvicorn-0.39.0-py3-none-any.whl (68 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68.5/68.5 kB 8.1 MB/s eta 0:00:00
Collecting mlflow
  Downloading mlflow-3.1.4-py3-none-any.whl (24.7 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.7/24.7 MB 5.0 MB/s eta 0:00:00
Collecting scikit-learn
```

```
hadoop@BDTT:~/fraud-detection-demo$ docker run --name iris-service -p 8080:8080 iris-model:final
2026/03/27 05:46:33 WARNING mlflow.utils.requirements_utils: Detected one or more mismatches between the model's dependencies and th
e current Python environment:
 - mlflow (current: 3.1.4, required: mlflow==3.10.1)
 - numpy (current: 2.0.2, required: numpy==2.2.6)
 - psutil (current: uninstalled, required: psutil==7.2.2)
 - pyarrow (current: 20.0.0, required: pyarrow==23.0.1)
 - scikit-learn (current: 1.6.1, required: scikit-learn==1.7.2)
 - scipy (current: 1.13.1, required: scipy==1.15.3)
To fix the mismatches, call `mlflow.pyfunc.get_model_dependencies(model_uri)` to fetch the model's environment and install dependenc
ies using the resulting environment file.
2026/03/27 05:46:33 WARNING mlflow.pyfunc: The version of Python that the model was saved in, `Python 3.10.12`, differs from the ver
sion of Python that is currently running, `Python 3.9.25`, and may be incompatible
/usr/local/lib/python3.9/site-packages/sklearn/base.py:380: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeCla
ssifier from version 1.7.2 when using version 1.6.1. This might lead to breaking code or invalid results. Use at your own risk. For
more info please refer to:
```

Output:

```
hadoop@BDTT:~/fraud-detection-demo$ curl -X POST http://localhost:8080/predict -H "Content-Type: application/json" -d '{"sepal_lengt
h":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
hadoop@BDTT:~/fraud-detection-demo$
```

Output in UI:



Curl

```
curl -X 'POST' \
  'http://localhost:8080/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}'
```

Request URL

```
http://localhost:8080/predict
```

Server response

Code	Details

200	**Response body**
	`{ "prediction": [ 0 ] }` Download
	Response headers
	`content-length: 18`
	`content-type: application/json`
	`date: Fri,27 Mar 2026 05:49:00 GMT`
	`server: uvicorn`

Responses

Code	Description	Links
200	Successful Response	No links